

nextATT&CKs

Pitch Brief — everything, basic to advanced

ET AI Hackathon 2026

Problem Statement 7 — AI-Driven Cyber Resilience for Critical National Infrastructure

Every figure in this brief is produced by an evaluation script in the repository. Nothing is rounded up and nothing is aspirational. Where something was not measured, this document says so in the same words the product does.

Read Part 13 first if you are short of time.

nextATT&CKs — Pitch Brief

ET AI Hackathon 2026 · Problem Statement 7 · AI-Driven Cyber Resilience for Critical National Infrastructure

Everything in this document is a number the repository can produce on demand.

Nothing is rounded up, nothing is aspirational. If a figure is here, you can defend it; if we did not measure something, this brief says so in the same words the product does.

PART 1 — THE THIRTY-SECOND VERSION

What it is

An AI-augmented Security Operations Centre for critical national infrastructure.

It takes a raw authentication or network log, finds the attack inside it, maps it to MITRE ATT&CK, shows how far the attacker can reach, forecasts what they do next, and proposes a containment action for a human to approve.

The one-sentence pitch

Most security AI tells you it is 94% confident. Ours tells you what it does not know, and can prove every number on screen.

Why we win on the thing nobody else does

Every team will show detection. The differentiator is **calibrated honesty**:

- A finding is labelled **observed** (in the logs) or **inferred** (derived, may be wrong) — never presented identically.
- A metric we did not measure says "**Not measured**" and why. Never a zero.
- We publish results **that go against us**. A logistic regression beats our detector on one measure. It is on the scoreboard.
- Two independent analysis lanes run on the same log. When they disagree, the screen says so and confidence *drops*.

That is the argument: in critical national infrastructure, a system that overstates gets someone killed. Ours is built so it cannot.

PART 2 — THE PROBLEM

What PS7 asks for

AI-driven cyber resilience for CNI: hospitals, exam boards, power, transport.

Detect, attribute, predict, respond — with explainability.

Why this is hard in India specifically

- **AIIMS Delhi, November 2022.** Ransomware took the hospital's systems down.

Patient care ran on paper for days.

- CNI operators have small SOC teams and enormous log volume.
- Most commercial tooling assumes cloud, licences and connectivity.

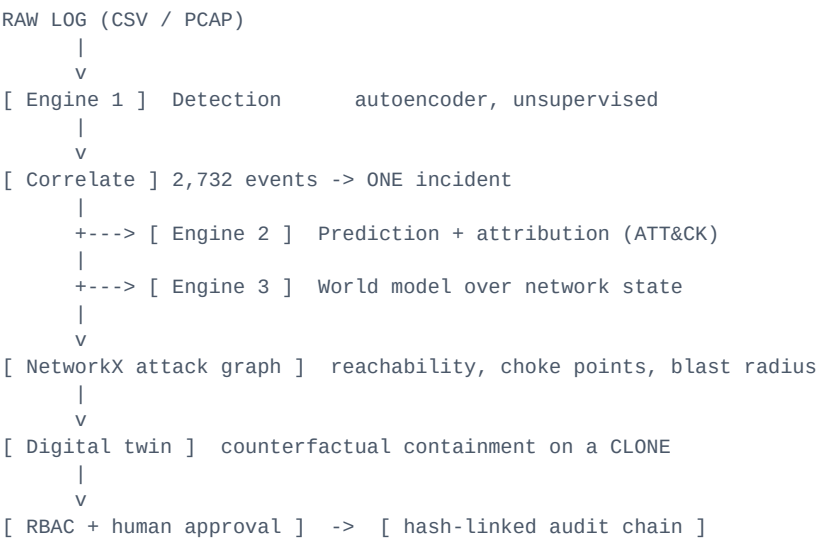
The constraint we set ourselves

Zero cost. No API key. Fully offline. One container.

Not a limitation we tolerated — a design requirement. A tool a district hospital cannot run is not a tool for Indian CNI. The whole product works with no credentials and no internet.

PART 3 — ARCHITECTURE

The shape



Two analysis lanes, one authority

This trips people up, so be ready for it:

	Workflow lane	Agent lane
Shape	7 bounded nodes, one replan	10 agents over behavioural chunks
Severity from	peak calibrated anomaly score	prioritiser risk band over ranked chains
Owns	claims, assessment, twin, RBAC, audit	chunking, per-window summaries, narrative
Authority	authoritative	cross-check only

The agent lane is a *second opinion*, not a second verdict. Its agreement adds confidence, capped at **0.45** — because the two lanes read the same log and share a rule table, so they are only *partially* independent. Treating them as two sensors would be double-counting.

If asked "why not just average them?" — because that invents a number neither analysis computed. Disagreement is signal, not noise to smooth away.

The stack

- **Backend:** Python, FastAPI, NumPy, scikit-learn, NetworkX
- **Frontend:** React 19 + TypeScript (strict), Vite, Tailwind v4, Radix, Framer Motion, three.js
- **Deploy:** one Docker container, ~30,651 lines of Python, 503 automated tests
- **Data:** MITRE ATT&CK (STIX), CISA KEV, CERT-In, NVD — all bundled offline

PART 4 — ENGINE 1: DETECTION

What it does

Finds anomalous authentication events with **no labels**. Trained only on benign traffic; anything that reconstructs badly is anomalous.

Why unsupervised

A novel campaign has no labels. If your detector needs them, it cannot catch what it has not already seen. This is the core argument for the design.

The measured results

Dataset	Metric	Result
LANL (real red-team)	ROC-AUC	0.992
LANL	TPR @ 1% FPR	87.7% (616 of 702 caught)
LANL	TPR @ 5% FPR	96.6%
LANL	Behavioural-only ROC (NTLM removed)	0.906
CIC-IDS2017	PR-AUC, autoencoder	0.570
CIC-IDS2017	PR-AUC, IsolationForest	0.473
CIC-IDS2017	PR-AUC, rule baseline	0.098 (worse than random)
CIC-IDS2017	PR-AUC, random	0.155
UNSW-NB15	ROC-AUC	0.829

We report PR-AUC and TPR at fixed FPR, never accuracy. Attack prevalence is 0.006%. A model that says "benign" every time is 99.994% accurate and useless.

Say this line — it shows you understand the metric.

Why the autoencoder over IsolationForest

ROC barely separates them (0.992 vs 0.988). The deciding number is the operating point an analyst actually runs at: at 1% FPR the autoencoder catches ****616 of 702, the forest 361 of 702****.

The NTLM ablation — the honesty set-piece

100% of red-team logins used NTLM; ~6% of benign did. That is a powerful signal and an **evadable** one. So we removed it and re-measured: ROC-AUC **0.906** on behaviour alone.

Why this matters in a pitch: it proves detection is driven by generalisable behaviour, not one brittle artifact. Most teams would never run this test

because it can only make their number look worse.

The result that goes against us

A **supervised logistic regression** on the identical seven features beats us:

	LR (supervised)	Our autoencoder
TPR @ 1% FPR	0.9194	0.9005
PR-AUC	0.0878	0.0088

Three qualifiers, and know all three:

1. LR trains **with the red-team labels**. A novel campaign has none.
2. The stratified split puts the same campaign on both sides — that flatters a supervised model far more than an unsupervised one.
3. **At a usable threshold LR is unusable**: F1 of 0.004 at a 3.1% false-positive rate. It ranks well and cannot be operated.

If a judge raises it first, you have lost the moment. Raise it yourself.

"We benchmarked against the supervised baseline the problem statement names, and it beat us on ranking. Here is why we still ship the unsupervised one."

The seven features

is_fail, new_dst_for_user, new_src_for_user, user_distinct_dst_sofar,
user_fail_rate_sofar, dst_rarity, is_ntlm

PART 5 — ENGINE 2: PREDICTION AND ATTRIBUTION

Next-technique prediction

Interpolated Markov model over **205 real ATT&CK campaign sequences**

(order-2 + order-1 + unigram, weights 0.2 / 0.3 / 0.5).

Method	Top-3 accuracy
Most-frequent baseline	4.9%
Kill-chain order baseline	7.1%
LSTM over MiniLM embeddings	27.2% — lost, published
Markov, first order	36.5%
Interpolated Markov (shipped)	38.1%

The anti-circularity argument — learn this one

The sequences are tactic-ordered, so a model could score well by simply re-learning "reconnaissance comes before exfiltration" and learning nothing about attacks. So we **built a baseline designed to beat us**: a kill-chain order model. It gets 7.1%. We get 38.1% — **5.4x** — which is evidence we predict real technique-to-technique transitions, not the ordering convention.

Being able to say "we built a baseline specifically to catch ourselves cheating" is worth more than the accuracy number.

The published negative

An LSTM lost at 27.2%. A bidirectional LSTM got 20.0%. Both are in `reports/model_experiments.md`. We ship the simpler winner. Neural is not automatically better at this data scale, and we can prove we checked.

Non-circular India test

On 4 analyst-verified CERT-In sequences ordered by the ****real reported timeline rather than our heuristic: top-3 is 10%****, against 38.1% on the auto-ordered set. Real orderings are harder. We publish both.

Attribution — say this carefully

Transparent weighted retrieval over **172 MITRE group profiles**

(coverage 0.55, Jaccard 0.20, semantic similarity 0.25), with a printed justification.

It is not a trained classifier and we never call it one. The screen shows ranked *candidates* with the overlap that produced each score. "APT28 uses pass-the-hash" is a statement about the technique, never an attribution of this incident to APT28.

PART 6 — ENGINE 3: THE WORLD MODEL

Built for the SIH 2026 "World Models" problem statement. This is the most technically advanced piece.

The distinction that matters

Engine 2 learns $P(\text{technique} \mid \text{previous techniques})$ — transitions between labels an analyst assigns. Engine 3 learns $P(S_{t+1} \mid S_t)$ where S_t is observed network state. That is a different and harder question.

How it works

1. **State:** 48 dimensions per window of 256 consecutive CIC-IDS2017 flows — TCP flag distribution, IAT statistics, bidirectional ratios, packet-length distribution, TCP window sizes, throughput. Mean *and* standard deviation of each, so the vector carries dispersion, not just centre.
2. **Quantise:** k-means into **24 latent states**. The state is inspectable — you can print what state 7 means.
3. **Dynamics:** a 24×24 Laplace-smoothed transition matrix, counted per day so no edge crosses a day boundary.
4. **Forecast:** $p_{\theta} @ T^k$. Exact matrix rollout, not sampling — deterministic, no sampling error.

Trained Mon–Wed, tested Thu–Fri. **Temporal split**, so no attack burst appears on both sides.

Results — and the split is the story

Where it wins:

Metric	Model	Baseline
Next-window compromise ROC-AUC	0.9872	0.5 (random)
PR-AUC	0.9333	
Attack-rate Brier @ 1 step	0.02217	0.12353 — 5.6× better

Where it only draws:

Next-state top-1	
Counted matrix alone	0.2733
Persistence baseline ("assume no change")	0.3620
Offline model	0.3567
Online adaptive (shipped)	0.3964
Oracle (cheats, counted on test days)	0.4475

The three attempts — this is the best engineering story you have

1. **Second-order model.** The obvious fix: an order-1 matrix cannot tell "we have been sitting in state B" from "we just arrived in B from A". It was **worse** — 0.2357 alone, and leave-one-day-out gave it weight **zero**.

Momentum was not the missing ingredient.

2. **We built an oracle on purpose.** A matrix counted on the *test* days themselves reaches **0.4475**, beating persistence by 8.6 points. That proved a first-order model *can* win — so the limit was **transfer** between days, not model capacity^{**}. Different problem, different fix.

3. **Causal online adaptation.** Transfer is fixable at deployment with **no labels^{**}**: traffic arrives, you observe its transitions, so you may count them. The tracker predicts, *then* is told what happened. **0.3964 vs 0.3620** — and it sits below the oracle, where an honest causal model must.

^{*}A test asserts a prediction at window *i* is byte-identical whether or not windows after *i* exist. If future evidence ever leaks in, it fails loudly instead of quietly improving the number.^{*}

Honest framing

"A strong risk model over network state, and a mediocre state forecaster.

We know exactly how mediocre and we published it."

PART 7 — PACKET-LEVEL AND PCAP

30 packet features covering everything the problem statement names:

- **TTL:** mean, variance, cardinality
- **TCP window:** mean, std, zero-rate
- **Fragmentation:** frag rate, don't-fragment, more-fragments
- **Payload:** mean, std, zero-rate, **Shannon entropy**
- **Port-scan signature:** unique dst ports, ports-per-host, SYN-without-ACK
- **Retransmissions:** repeated (flow, seq) pairs carrying payload

Two readers, and the plain one is default. Classic PCAP parses with struct alone — so the slim deployed image keeps every packet feature with ****no new dependency****. Scapy is used when installed, for pcapng and awkward link types.

The bug the cross-check caught

Both readers must produce identical features from the same file. First run they disagreed: 300 unique destination ports from one, 282 from the other. The stdlib reader was parsing the leading bytes of a **non-first IP fragment** as a TCP header — inventing ports and sequence numbers out of payload continuation, and manufacturing retransmissions from them. Scapy was right.

No detection accuracy is claimed. No labelled capture ships with the repo. What is verified is that each feature computes what it claims, against frames whose properties the test chose. 29 tests.

PART 8 — THE HONESTY MACHINERY

This is your differentiator. Know it cold.

Evidence-calibrated claims

Every finding carries a status:

Status	Meaning
observed	In the logs. The strongest thing we can say.
confirmed	Corroborated against the public record.
inferred	Derived from behaviour. Could be wrong.
predicted	A forecast. Has not happened.
disputed	Two analyses disagree. Neither suppressed.

Only observed and confirmed are **actionable**. An inferred finding carries its missing evidence and its benign alternatives.

The T1078 story — tell this one

We originally mapped every anomalous login to T1078 (Valid Accounts). It lifted technique coverage from 37.5% to 100%. It was **wrong** — an anomalous login is not evidence of a stolen credential; it could be a new laptop or a night shift.

Now T1078 is *inferred*, strength 0.30, confidence 0.186, **not actionable**, with the missing evidence spelled out.

We made coverage look worse on purpose because the higher number was a lie.

Noisy-OR with independence groups

Confidence combines across **independence groups**. Two pieces of evidence from the same source add nothing — duplicates do not compound. This is why the agent lane is capped at 0.45.

Four scores, never one

Anomaly, likelihood, impact, confidence — reported **separately**. A single blended "risk score" hides which of four different things is high.

Exact Shapley values

Feature attribution with **all 128 coalitions enumerated** — seven features means exhaustive enumeration is cheap, so the values are **exact**, not approximated. No sampling error. The efficiency axiom is asserted at runtime.

Most teams will say "we use SHAP". You can say "we compute exact Shapley values because with seven features approximation is unnecessary".

Hash-linked audit chain

Every decision: `sha256(prev_hash + canonical_json)`. Tamper-evident. Verified and tamper-detection tested. Export and re-verify supported.

RBAC — server-enforced

Four roles: viewer / analyst / responder / admin. Enforced **server-side**.

Hiding a button is a courtesy, never the mechanism. Flip the role selector and watch the API refuse — the refusal text is the demo.

Everything is simulated

No response action ever executes against a real system.

`actions_executed_against_real_systems = 0`. The digital twin runs counterfactuals on a **clone** of the graph.

MTTR is "Not measured" — with nothing executed there is no repair to time.

Claiming an MTTR improvement would be fabricating the exact headline PS7 asks for. *This is the single strongest honesty moment in the whole product.*

PART 9 — THE AIIMS CASE FILE

The 2022 AIIMS Delhi ransomware incident, built **only** from verified parliamentary records.

- **Sources:** Rajya Sabha Q.1043 (10.02.2023), Lok Sabha Q.1837 (16.12.2022) — we fetched the PDFs and extracted the text ourselves rather than trusting a summary. A third source returned HTTP 403 and is marked **unverified**.

- **Only T1486 (Data Encrypted for Impact) is confirmed.** T1021 is inferred.

- **Nine things are listed as NOT established by the record** — including several commonly repeated in press coverage.

Why this wins: every other team will cite AIIMS from a news article. You can say which parliamentary answer each fact came from, and which widely-repeated "facts" the record does not support.

PART 10 — LIVE DEMO NUMBERS

The shipped LANL campaign scenario:

Events	2,732
Correlated alerts	1,243
Collapsed into	1 incident
Severity	critical
Accounts involved	104
ATT&CK chain	T1550.002 → T1078 → T1110 → T1021
Graph	473 nodes, 484 edges
Blast radius	469 hosts reachable

Performance

- Full 7-node investigation: **p50 790ms, p95 1,411ms** (laptop CPU, no GPU)
- Deterministic analysis alone: **185ms** on the full campaign
- Agent cross-check lane adds: **419ms** (AIIMS) / **1,910ms** (LANL)

Be honest about the trade: the authoritative answer is ready in under 200ms;
the second opinion costs ~2s and can be switched off.

Scoreboard

26 cards: 24 measured, 2 declared "Not measured" (technique precision, MTTR).

PART 11 — THE QUESTIONS YOU WILL GET

"Is this just a wrapper around ChatGPT?"

No. Every score, severity, technique ID, probability, graph traversal, hash and policy decision is deterministic Python. A language model is **optional**, **off by default**, and only ever *rewords* figures already computed. It cannot produce a number or approve an action. `LLMResult.authoritative` cannot even be set to true through the constructor.

"What is your accuracy?"

Reframe: at 0.006% attack prevalence, accuracy is meaningless — predicting "benign" always scores 99.994%. We report ROC-AUC 0.992 and TPR 87.7% at a 1% false-positive rate, which is the operating point an analyst runs at.

"How is this different from Splunk / Wazuh / a SIEM?"

A SIEM tells you what happened. This tells you what it *cannot* establish, forecasts what happens next, and simulates the containment before you approve it — with every claim labelled by how strongly it is evidenced.

"Does it work on our data?"

It ingests CSV and PCAP. Three public datasets are benchmarked. The rule map has 9 event types; anything outside is reported **unmapped rather than guessed**.

"What is the false-positive rate?"

Calibrated to a 1% FPR operating point, catching 87.7% of red-team events. The calibration is piecewise-log: $p_{50} \rightarrow 0$, $p_{99} \rightarrow 50$, so 50 is the 1% FPR line.

"Why should we trust the AI?"

You should not, and the product is built on that assumption. That is why nothing executes, why every claim is labelled, why a missing metric says "Not measured", and why we publish the benchmark that beats us.

"What does not work?"

- No measured detection accuracy on packet data (no labelled capture bundled)
- Not tested on CTU-13 or CIC-IDS2018
- Engine 3's next-state forecast only draws with a persistence baseline
- Technique precision is not measurable — no public dataset carries per-event ATT&CK labels
- MTTR is not measured, because nothing executes

Having this list ready is a strength. Fumbling it is the only way it hurts.

PART 12 — DEMO SCRIPT

1. **Investigation tab** — run the AIIMS scenario. Watch the 7 nodes complete.

Point at the stage rail: *"every stage is bounded and traced."*

2. **Findings** — point at claim status. *"This one is observed. This one is inferred, and here is the evidence we are missing."*

3. **Assessment** — four separate numbers. *"We never blend these."*

4. **Cross-check** — two lanes, their severities, the disagreement.

5. **Attack graph (3D)** — entry host, pivots, crown jewels, choke points.

6. **Digital twin** — simulate isolating the recommended host. *"On a clone. Nothing has happened to the network."*

7. **Approve** — flip role to Viewer, try to approve, **show the refusal**.

Flip to Responder, approve, show the audit hash.

8. **Scoreboard** — scroll to a "Not measured" card and to the LR baseline we lose. *"We publish the ones that go against us."*

Close on the scoreboard. It is the argument.

PART 13 — NUMBERS TO MEMORISE

If you remember nothing else:

- **0.992** — LANL ROC-AUC
 - **87.7%** — TPR at 1% false-positive rate (616 of 702)
 - **0.906** — ROC with NTLM removed (the ablation)
 - **38.1% vs 7.1%** — prediction vs the baseline built to beat us (5.4×)
 - **0.9872** — next-window compromise ROC-AUC (Engine 3)
 - **2,732** → **1** — events collapsed into one incident
 - **469** — hosts in the blast radius
 - **503** — automated tests, all offline
 - **24 of 26** — scoreboard cards measured; 2 say "Not measured"
 - **0** — response actions executed against real systems
 - **Rs 0** — cost to run: no API key, no cloud account, no licence
-

The last line

"Every number on that screen, we can show you how we got.

And the ones we could not measure, we left blank on purpose."